B Fitle

B.1 Overview

`fitle` is a lightweight Python framework for building statistical models, compiling them into efficient machine code with `Numba` [1], and fitting to data using `iMinuit` [2, 3]. `fitle` is available on github.

A typical workflow is:

1. Define a `Model` by doing symbolic arithmetic on components and fitting parameters
2. Pipe the model into a cost function such as Negative Log Likelihood (NLL) or χ^2, along with the data to be modeled;
3. Minimize the cost function using the `fit()` method, which wraps `iMinuit`.

The most important structure in the library is a `Model` that represents a mathematical expression which relates inputs (`INPUT`), constants, and parameters (`Param`s). The user rarely needs to define a `Model` explicitly. Performing any arithmetic operation or function on a `Param` will return a `Model` representing that expression. Examples are provided below, after the API description.

B.2 API Reference

```
# === Params ===
# Shorthand with unary operators (auto-named from variable)
a = ~Param                          # Unbounded param, auto-named 'a'
sigma = +Param                      # Positive (>0), auto-named 'sigma'
neg = -Param                        # Negative (<0), auto-named 'neg'

# Explicit creation with chaining
Param('name')                       # Named parameter
Param(start)                        # Set starting value
Param(min, max)                     # Set bounds
Param('name')(start)(min, max)      # Chaining in any order

# Positive param with start value
sigma = (+Param)(2.5)               # Positive, start=2.5, auto-named

# Factory methods
Param.positive(name=None)           # Bounds (1e-6, inf), start=1
Param.negative(name=None)           # Bounds (-inf, -1e-6), start=-1
Param.unit(name=None)               # Bounds [0,1], start=0.5

# Special params
INPUT                               # Placeholder for input data (x)
INDEX                               # Default index placeholder
index(*args)                        # Create INDEX param with range(*args)

# === Model Construction ===
```

```
const(value)                          # Wrap constant as Model
identity(val)                         # Wrap in identity function
gaussian(mu=None, sigma=None)         # Gaussian PDF
exponential(tau=None, start=None, end=None)
                                      # Exponential PDF (truncated if start/end given)
crystalball(alpha, n, mu, sigma)      # Crystal Ball PDF
convolve(d_x, c, mass_mother, mu, sigma, bin_width=1.0)
                                      # Discrete convolution

# === Indexing ===
model[INDEX]                          # Make model indexable by INDEX
model[i]                              # Evaluate/substitute at index i
const(arr)[INDEX]                     # Select element from array
INPUT[0]                              # Select element 0 from input

# === Model Properties ===
Model.params                          # List of THETA parameters
Model.free                            # List of free params (INPUT, INDEX)
Model.components                      # Sub-models split at iden() boundaries
Model.compiled                        # True if numba-compiled version exists

# === Model Methods ===
Model.__call__(x=None)                # Evaluate model; x required if INPUT free
Model.__mod__(subs)                   # Substitute: model % {param: value}
Model.__getitem__(map)                # Index substitution: model[i]
Model.grad(wrt=None)                  # Symbolic gradient w.r.t. params
Model.simplify()                      # Algebraic simplification
Model.shape()                         # Infer output shape
Model.freeze()                        # Replace params with current values
Model.copy()                          # Deep copy of model tree
Model.compile()                       # JIT compile; stores in cache

# === Reduction ===
Reduction(model, index_param, op=operator.add)
                                      # Reduce model over index with operator
                                      # Example: sum over n bins

# === Cost Functions ===
Cost.MSE(x, y)                        # Mean squared error
Cost.NLL(data)                        # Unbinned negative log-likelihood
Cost.binnedNLL(data, bins, range)     # Binned NLL
Cost.chi2(data, bins, range)          # Chi-squared (binned)
Cost.chi2(x=centers, y=counts)        # Chi-squared from histogram
Cost.bin_widths                       # Bin widths (if binned method)

# === Fitting ===
fit(model, grad=True, ncall=9999999, options={})
                                      # Returns FitResult
```

```
FitResult.fval                    # Best objective value
FitResult.success                 # Convergence flag
FitResult.values                  # Dict of {name: fitted_value}
FitResult.errors                  # Dict of {name: error}
FitResult.predict                 # Frozen model scaled by bin_widths
FitResult.minimizer               # Underlying iminuit.Minuit object
```

B.3 Example: Linear Fit

First, we can define parameters using `Param`. The shorthand `+Param` creates a positive parameter that is automatically named from the variable it is assigned to. Calling a `Param` with two scalars sets its bounds, with one scalar sets its starting value, and with a string sets its name. This can be done in any order. We define a `Model` that represents the linear equation $y(x) = a \cdot x + b$, where $a > 0$ and b is between -5 and 5 with

```
    from fitle import Param, INPUT, fit, Cost

    a = +Param                          # Positive, auto-named 'a'
    b = Param(-5, 5)                 # Bounded [-5,5], auto-named 'b'

    linear_model = a * INPUT + b
```

Here, `linear_model` represents $y(x) = a \cdot x + b$ and `linear_model(x)` returns the output of that with the starting values of the parameters. To fit this equation to data, we define a `Cost` instance with the data, where we assume two `ndarray`'s `data_x` and `data_y` and the method, which is Mean Squared Error (MSE).

```
    import numpy as np

    data_x = np.array([-2,-1,0,1,2])
    data_y = np.array([0,0,1,1,2])

    cost = Cost.MSE(data_x, data_y)
```

With the pipe operator, we create a `Model` that represents the cost of `linear_model` with respect to the data and method.

```
    model_cost = linear_model | cost
    model_cost
    # sum(([0 0 1 1 2] - (a=1 * [-2 -1  0  1  2] + b=0)) ** 2)
```

As seen above, `model_cost` is a `Model` that represents the MSE of `linear_model` on the data and `model_cost()` returns the cost metric with the starting values of `a` and `b`, as in

```
    model_cost()
    # 6.0
```

To fit, one can write

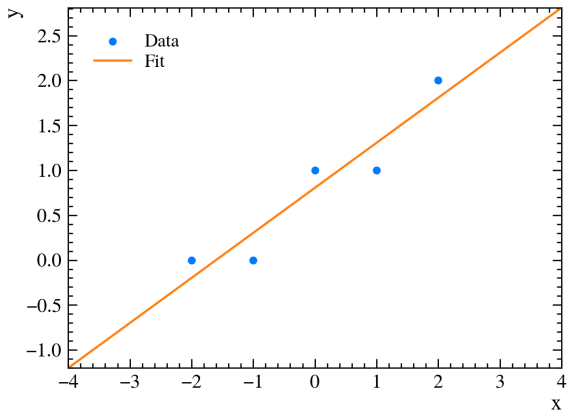
```
    fit_result = fit(model_cost)
    fit_result
```

```
    # <<FitResult fval=0.300, success=True>
    # a: 0.5015 ± 0.31
    # b: 0.8039 ± 0.45
```

By default, this calculates the gradient of the cost with respect to `a` and `b`, and compiles both the cost function and its gradient using `numba` which is probably overkill for this simple model. Plotting the data and printing the fit results are easy:

```
    import matplotlib.pyplot as plt

    plt.plot(data_x, data_y)
    plt.plot(data_x, linear_model(data_x))
```




```
    print(a.value, a.error)
    # 0.501474455674885 0.3104269246452036
    print(fit_result.errors)
    # {'a': 0.3104269246452036, 'b': 0.44663075536333074}
```

This works because the values of `a` and `b` are changed to those that minimize the cost function. The same fitting code may be written more concisely as:

```
    fit_result = fit((+Param)('a') * INPUT + Param('b')(-5, 5) |
       Cost.MSE(data_x, data_y))
```

B.4 Example: Double Gaussian Fit

The `gaussian` method in the library is functionally equivalent to

```
    def gaussian(mu, sigma):
        norm = 1 / (sigma * sqrt(2*pi))
        arg = -0.5 * ((INPUT - mu) / sigma) ** 2
        return norm * np.exp(arg)
```

The `exponential` is functionally equivalent to:

```
    def exponential(tau, start=None, end=None):
        if start is not None and end is not None:
            norm = 1 - np.exp(-(end - start) / tau)
```

```
            return (1 / tau) * np.exp(-(INPUT - start) / tau) / norm
        return (1 / tau) * np.exp(-INPUT / tau)
```

To fit two peaks as double gaussian signals (with common mean values) and an exponential background as in Fig. **??**, one can write:

```
from fitle import Param, gaussian, exponential, fit, Cost

Dp_mass = Param(1850)
Ds_mass = Param(1950)
Dp_signal = +Param * gaussian(mu=Dp_mass) + +Param * gaussian(mu
   =Dp_mass)
Ds_signal = +Param * gaussian(mu=Ds_mass) + +Param * gaussian(mu
   =Ds_mass)
background = +Param * exponential(tau=+Param)

model = Dp_signal + Ds_signal + background
```

One can fit this `Model` to the χ^2 of a histogram of 200 bins. Let `data` be an `ndarray` of mass values.

```
fit_result = fit(model | Cost.chi2(data, 200))
fit_result # inspect the fit
```

The data, the full fit projection, and projections of the fit components can be plotted separately. In the code snippet below, `x` represents the center of each bin in the histogram of `data`, and `y` the number of counts in a bin.

```
plt.plot(x, y)
plt.plot(x, model(x))
plt.plot(x, Dp_signal(x))
plt.plot(x, Ds_signal(x))
plt.plot(x, background(x))
```

B.5 Crystal Ball Function

The Crystal Ball function combines a Gaussian core with a power-law tail:

$$f_{\mathrm{CB}}(x;\alpha,n,\mu,\sigma) = N \cdot \begin{cases} A\left(B - \frac{x-\mu}{\sigma}\right)^{-n}, & \frac{x-\mu}{\sigma} < -\alpha, \\ \exp\left(-\frac{1}{2}(\frac{x-\mu}{\sigma})^2\right), & \frac{x-\mu}{\sigma} \geq -\alpha, \end{cases} \tag{B.1}$$

The `fitle` code is functionally equivalent to:

```
def crystalball(alpha, n, mu, sigma):
    n_over_alpha = n/alpha
    exp = np.exp(-0.5*alpha ** 2)
    A = (n_over_alpha)**n * exp
    B = n_over_alpha - alpha
    C = n_over_alpha/(n-1) * exp
    D = np.sqrt(0.5*np.pi) * (1 + Model(erf, [alpha/np.sqrt(2)])
        )
```

```
        N = 1/(sigma*(C + D))

        z = (INPUT - mu)/sigma

        return np.where(z > -alpha,
                        N * np.exp(-0.5*z**2),
                        N * A * (B - z)**-n
                       )
```

Here we see the representation of conditions with `np.where(z > -alpha)`, and the base constructor for `Model` that takes a function and a list of arguments. It is also worth noting that wrapping `scipy.special.erf` in a lambda prevents differentiation and compilation.

B.6 Example: Convolutional Fit

The key method in this work was to convolve QED-based radiative predictions (PHOTOS simulations) with the approximate resolution function.

$$(Q \star R)(x) \approx \sum_{n=0}^{n_{\text{bins}}-1} Q(x'_n)\, R(x - x'_n)\, \Delta x'_n \tag{B.2}$$

where Q denotes the radiative distribution and R the resolution function (e.g. double Gaussian). In `fitle`, discrete convolution is implemented with indices and reductions in a way equivalent to:

```
    def convolve(centers, counts, mass_mother, mu, sigma, bin_width
       =1.0):
        n = index(len(counts))
        Qx_n = const(counts)[n]
        x_n = const(centers)[n]
        shifted_x = INPUT + mass_mother - mu
        R = gaussian(mu=x_n, sigma=sigma) % shifted_x
        weighted = Qx_n * R
        ret = Reduction(weighted, n, operator.add)
        return ret / (bin_width * np.sum(ret))
```

In this function, `index` creates `n` which is an index parameter, a kind of `Param` that stores ranges. The indexing syntax `const(counts)[n]` creates a `Model` that represents the `n`th count of a histogram (such as a PHOTOS histogram); similarly `const(centers)[n]` selects the `n`th center. The `%` operator replaces elements in a model with a map, but in the absence of a dictionary, as in the code above, we replace `INPUT` by default. The $\Delta x'_n$ term of Eq. B.2 is not included explicitly as it is already included in the `counts` of the PHOTOS histogram. The `Reduction` of `weighted` over `n` using addition represents the summation in Eq. B.2. Finally, we normalize by dividing by the bin width and the sum of the result.

References

[1] S.K. Lam, A. Pitrou and S. Seibert, *Numba: A llvm-based python jit compiler*, in *Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC*, pp. 1–6, 2015.

[2] H. Dembinski and P.O. et al., *scikit-hep/iminuit*, .

[3] H. Dembinski, P. Ongmongkolkul et al., *scikit-hep/iminuit*, *Journal of Open Source Software* **5** (2020) 2361.